Listen

Learning Doc 08: The Web App
React, TypeScript, pages, data and charts

Detailed design, Section 7 (visual design follows in Figma)

17 October 2026

Abstract

This document designs the browser side of Listen: what a React app is, how TypeScript keeps it in step with the server, how pages get their data, what each page shows, and which chart answers which question, including the calendar heatmap. The *look* of the pages (layout, spacing, exact colours on screen) will be designed in Figma and added as a separate part once the Figma connection is available; this document fixes everything the look must serve.

Contents

1 Decisions this section is built on **2**

2 What a React app is **2**

3 TypeScript in this app **3**

4 Project structure of `web/` **4**

5 Moving between pages, and staying logged in **4**

6 Getting data: one cache for the whole app **5**

7 The pages **5**
- 7.1 1. Login . . . 5
- 7.2 2. Overview, “today at a glance” . . . 5
- 7.3 3. History . . . 5
- 7.4 4. App usage . . . 5
- 7.5 5. Commands . . . 5
- 7.6 6. Insights . . . 5
- 7.7 7. Devices & settings . . . 6

8 Charts: the right form for each question **6**
- 8.1 The activity calendar . . . 6
- 8.2 Rules every chart follows . . . 7

9 Visual design in Figma (next) **7**

10 Testing the web app **7**

11 Glossary **7**

12 Check your understanding **8**

1 Decisions this section is built on

Topic	Decision
Language	**TypeScript**, with API types generated from the server.
Screens	Laptop only. Pages are designed for a wide window (1280 px and up); no phone layout.
Pages	Login, Overview, History, App usage, Commands, Insights, Devices & settings. Build order 1→4 first.
Activity calendar	A calendar heatmap on Insights; a square's darkness = number of commands that day.
Visual design	In Figma (pending connection).
From earlier	Talks only to the API (`/v1/...`); refreshes every 5 minutes; guided form for command edits (Doc 07).

2 What a React app is

Concept: Single-page application (SPA)

A traditional website asks the server for a whole new page on every click. A **single-page application** loads *once*: the browser downloads one bundle of JavaScript, which then draws every page itself and only asks the server for *data* (JSON from the API). Clicking from Overview to History swaps what is on screen instantly, without reloading.

An example: opening Listen, then clicking History.

Moment	Traditional website	Listen (SPA)
You open the site	Server sends a full Overview page (HTML)	Server sends one small HTML file + the app's JavaScript bundle (once); the app draws Overview, then fetches `/v1/analytics/summary` (JSON)
You click "History"	Browser throws the page away and downloads a full new History page; the screen flashes white	The app changes the address to `/history`, swaps the Overview component for the History component, fetches `/v1/analytics/commands` (JSON). No reload, no flash
You click "Overview" again	Another full page download	Overview's data is still in the cache (TanStack Query): it appears instantly

Gmail, Google Maps, Figma and Spotify's web player all work this way: after the first load, only data travels.

Concept: Component

React builds the screen out of **components**: functions that take some inputs and return what to show. A page is a component made of smaller components (a stat tile, a chart, a table row), the same way Listen's agent is made of small modules.

```
// web/src/components/StatTile.tsx   (TypeScript + JSX = "TSX")
type StatTileProps = { label: string; value: number; unit?: string };

export function StatTile({ label, value, unit }: StatTileProps) {
  return (
    <div className="stat-tile">
      <span className="label">{label}</span>
      <span className="value">{value}{unit}</span>
    </div>
  );
}

// used on the Overview page:
<StatTile label="Commands today" value={42} />
```

`label`, `value` and `unit` are the component's **props** (its inputs); the `?` on `unit` means it is optional.

Concept: JSX, and how TypeScript turns it into TSX

JSX lets you write HTML-looking tags *inside* JavaScript. It is not HTML and the browser cannot read it: before the app runs, the build tool (Vite) translates every tag into an ordinary function call. This line

```
<StatTile label="Commands today" value={42} />
```

becomes, roughly,

```
jsx(StatTile, { label: "Commands today", value: 42 })
```

So a tag is just a neat way to call a component function with its props. Curly braces `{...}` mean "this part is code, not text": `value={42}` passes the number 42, `value="42"` would pass the text `"42"`.
A file of *JavaScript* with JSX is `.jsx`; a file of *TypeScript* with JSX is **`.tsx`**. Same tags, but because the props have types, the tags are checked too:

```
<StatTile label="Commands today" value="42" />
// error: Type 'string' is not assignable to type 'number'.
<StatTile label="Commands today" />
// error: Property 'value' is missing.                          (value is required)
```

TSX = JSX's convenient tags + TypeScript's checking of every tag's props.

Concept: State and re-rendering

State is data a component remembers and may change, such as which date range is selected. When state changes, React runs the component again and updates only the parts of the screen that changed. You never edit the page by hand; you change state and React redraws.

3 TypeScript in this app

TypeScript adds type labels to JavaScript and checks them in the editor; before the browser runs anything, the labels are removed. Its biggest job here is keeping the web app and the server in agreement.

Concept: Generated API types

FastAPI publishes a machine-readable description of every endpoint at `/openapi.json` (the same data behind `/docs`). A tool (`openapi-typescript`) reads it and writes `web/src/api/schema.ts`: a TypeScript type for every request and response. One command regenerates it whenever the server changes. If a field is renamed on the server, every place in the web app that uses the old name turns red, and the app will not build until it is fixed.

```
// generated from the server; never edited by hand
type CommandRow = components["schemas"]["CommandRow"];
//   { uuid: string; ts: string; heard: string; result: "executed" | "failed" | ... }

row.heared      // error: Property 'heared' does not exist. Did you mean 'heard'?
```

Why it is done this way

Pydantic makes the *server* reject bad data; the generated types make the *web app* unable to misread good data. The server's Python models become the single source of truth for both halves, the same idea as `shared/recipe.schema.json` for commands (Doc 02).

4 Project structure of `web/`

```
web/
|-- package.json        <- dependencies + scripts (npm run dev / build / test / gen:api)
|-- vite.config.ts      <- Vite: the dev server and build tool
|-- src/
|   |-- main.tsx         <- entry point: mounts the app into the page
|   |-- App.tsx          <- routes: which URL shows which page
|   |-- api/             <- schema.ts (generated) + client.ts (one place that calls fetch
    )
|   |-- pages/           <- one file per page: Overview.tsx, History.tsx ...
|   |-- components/      <- reusable pieces: StatTile, DataTable, DateRangePicker ...
|   |-- charts/          <- CalendarHeatmap, TrendLine, AppBars ...
|   `-- styles/          <- colour tokens (light/dark) and base styles
`-- tests/
```

Concept: Vite

Vite is the tool that runs the app while you develop (`npm run dev`: saves show up in the browser instantly) and builds the final optimised files for deployment (`npm run build`). It handles TypeScript out of the box.

5 Moving between pages, and staying logged in

Concept: Client-side routing

React Router maps URLs to pages inside the SPA: `/` is Overview, `/history` is History, and so on. The address bar and the back button work as on any website, but no page is fetched from the server.

Protected pages. On start-up the app calls `GET /v1/auth/me`. If the answer is 401 (not logged in), every page redirects to `/login`. After logging in, the session cookie (Doc 07) does the rest; the web app never stores your password or the session id itself.

6 Getting data: one cache for the whole app

Concept: Server-state cache (TanStack Query)

Data from the API is stored in one shared **cache**, managed by a library called **TanStack Query**. A component says "I need `/v1/analytics/apps` for this week"; the library fetches it once, shares it with every component that asks for the same thing, refreshes it every 5 minutes (the decided refresh rate), and refreshes when you come back to the tab. Each request is in one of three states, and every page designs all three: **loading**, **error**, or **data**.

The sleeping server. If the first request takes longer than 3 seconds, the loading state changes to "Waking up the server… (this can take up to a minute on the free tier)" instead of an endless spinner.
One date range. A date-range control at the top of each analytics page (today, last 7 / 30 / 90 days, custom) drives every chart on that page together.

7 The pages

7.1 1. Login

Username, password, "Log in". Wrong password: one generic message ("Username or password is incorrect") that never reveals which one was wrong. Lock-out (Doc 07): "Too many attempts. Try again in 15 minutes."

7.2 2. Overview, "today at a glance"

- A **KPI row** of stat tiles: commands today, success rate today, top app today, median response time; each with its change versus the previous 7-day average and a small trend line (sparkline).
- The last 10 commands (what was heard, which command, result, time).
- Laptop status: "Last synced 3 minutes ago" (from `GET /v1/devices`).

7.3 3. History

Every command, newest first, 50 per page, with search on the heard text and filters (result, mode, command, date range). Each row can be marked *right* / *wrong* or deleted (Doc 07), with a confirmation for delete.

7.4 4. App usage

Time per app for the selected range, time per site inside a chosen app (Chrome → Google Search, YouTube …), and how one app's daily time changed over the range.

7.5 5. Commands

Your commands and routines as cards grouped by file (`core`, `browser`, `routines` …), read from the latest snapshot. "Add" and "Edit" open the guided form, which submits a pending edit; a panel lists pending edits with their status (waiting for laptop / applied / invalid / overridden).

7.6 6. Insights

The activity calendar, trends, sentences that did not match, latency, and calibration (charts below).

7.7 7. Devices & settings

Your laptop with its last sync time; create a new device token (shown once, with a copy button and a clear warning that it will not be shown again); revoke; log out.

8 Charts: the right form for each question

A chart is chosen by the *job* the reader must do, before any colour is picked.

Question	Form	Colour job
How active was I each day this year?	Calendar heatmap	Sequential (one hue; darker = more in light mode, brighter = more in dark)
Am I using Listen more or less over time?	Line, commands per day	One series: one hue, no legend
Is Listen getting things right more often?	Line, success rate per day	One hue
Today's headline numbers	Stat tiles, not charts	Ink only; delta arrow + label
Which apps take my time?	Horizontal bars, top 7 + "Other"	One hue (it is magnitude, not identity)
How did one app's time change?	Line: that app highlighted, others grey	Emphasis: one hue + grey
Where does the 1.5 s go?	Stacked bars per day, 4 stages, with a 1.5 s reference line	Categorical, 4 slots, direct-labelled
Are low scores really the wrong guesses?	Two small histograms (right / wrong) with the threshold marked	Two categorical slots
Which sentences failed?	A table (not a chart)	Ink only

8.1 The activity calendar

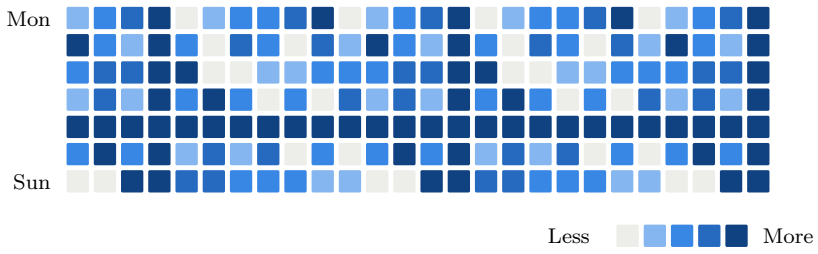


Illustration only (made-up data, 26 weeks shown); the real one shows 53 weeks.

- One square per day, columns are weeks, rows are weekdays.
- **Light mode: darker = more commands that day** (decided). Days with no commands are a light grey square, so "nothing" never looks like "a little".
- **Dark mode: brighter = more** (decided). On a dark page, "darker" would mean "closer to the background", so the busiest days would almost vanish. Instead busy days glow: zero is a dark slate square (`#2A3440`) and the 4 steps run `#1C5CAB` → `#2A78D6` → `#5598E7` → `#9EC5F4`, starting one shade lighter than light mode because the darkest blue failed the 2:1 contrast check against the dark surface. The rule in both modes: *more activity = stands out more.*

- Days with commands fall into **4 blue steps** using **quartiles** of your own active days, so one unusually busy day does not make every other day look empty.
- Hovering a square shows the date and the exact count; a "Less → More" key sits underneath. Data comes from a new endpoint, `GET /v1/analytics/calendar?year=2026`, returning one count per day.

Concept: Sequential vs. categorical colour

Sequential colour shows *how much*: one hue, light to dark (the calendar, app bars). **Categorical** colour shows *which one*: different hues for different things (the four latency stages). Using many hues for "how much" makes people read categories that are not there, so magnitude always uses one hue.

8.2 Rules every chart follows

- **One y-axis per chart.** Two measures with different units get two charts, never a double-axis chart (the most common way charts mislead).
- **Hover tooltips** on every chart: exact value, date, and label.
- **Never colour alone**: two or more series always have a legend, and up to four are labelled directly on the chart; every chart can switch to a table view.
- **Text stays in text colours** (black/grey ink), never in a series colour.
- **Validated palette**: a tested colour-blind-safe palette, checked with a palette validator for both light and dark mode. Status colours (good / warning / error) are reserved and always come with an icon and a word.
- **Light and dark mode**, following Windows' setting, each with its own validated steps.

Tools. **Recharts** (a React chart library) for lines and bars. The calendar heatmap is a small custom component drawn in SVG: 371 squares is simple enough that a library would add weight without adding value.

9 Visual design in Figma (next)

Once Figma is connected, each page gets a frame at laptop width (1440 px), using the colour tokens and chart rules above: Login, Overview, History, App usage, Commands, Insights, Devices. Those frames become the visual reference for building the pages and are added to this section.

10 Testing the web app

- **Type check** (`tsc`): the whole app must compile without type errors.
- **Component tests** with Vitest and React Testing Library: render a component with fake data and check what a user would see ("the calendar shows 371 squares", "a 401 sends you to the login page").
- The API is **mocked** in these tests, so they run in seconds without a server.

11 Glossary

Component / props
: A function that returns what to show / its inputs.

Calendar heatmap
: One square per day; darker means more (brighter in dark mode).

Generated types
TypeScript types produced automatically from the server's API description.

JSX / TSX
HTML-like syntax inside JavaScript / TypeScript code.

Quartiles
Values splitting sorted data into four equal-sized groups.

React Router
Maps URLs to pages inside a single-page app.

Sequential / categorical colour
One hue for "how much" / different hues for "which one".

SPA A web app that loads once and then only fetches data.

State
Data a component remembers; changing it redraws the component.

TanStack Query
A library that fetches, caches and refreshes API data.

Vite The development server and build tool for the web app.

12 Check your understanding

Check yourself

1. What does the browser download once in an SPA, and what does it keep fetching after that?
2. A field is renamed on the server. Walk through how the web app finds out.
3. Why do days with zero commands get grey, not the lightest blue?
4. Why are the calendar's blue steps based on quartiles of *your* days rather than fixed numbers like 0–5, 5–10?
5. Why does "time per app" use one hue, while "latency per stage" uses four?
6. You want commands per day and average latency on one chart. What do you do instead?